

Cryptography - 14_10

Recap from last lecture: OWF $\implies$ PRG with stretch $l(\lambda) = \text{poly}(\lambda)$.

Today: Let's apply what we have learned to SKE. Simple idea:

$$\text{Enc}(k, m) = G(k) \oplus m = c$$

$$\text{Dec}(k, c) = G(k) \oplus c = m$$

It's like OTP, with a big difference that: $k \in \{0, 1\}^\lambda$ but $m \in \{0, 1\}^{\lambda+l} \forall l = \text{poly}$.

But what does it mean for the above SKE to be (computationally) secure?

Let's define first a simple notion; this is just a warm-up.

✓ Definition

Let $\Pi = (\text{Enc}, \text{Dec})$ be a SKE. We say Π is one-time computationally secure if:

$$\text{GAME}_{\Pi, \mathcal{A}}^{1-\text{time}}(\lambda, 0) \approx_c \text{GAME}_{\Pi, \mathcal{A}}^{1-\text{time}}(\lambda, 1)$$

Let's see the experiments:

We have $\text{GAME}_{\Pi, \mathcal{A}}^{1-\text{time}}(\lambda, b)$, with λ being the security parameter and b a bit to show two experiments.

We have an adversary $\mathcal{A}$ and a challenger $\mathcal{C}_{k \in K}$. We have that $\mathcal{A}$ outputs two messages, $m_0, m_1 \in \mathcal{M}$, and the challenger is going to encrypt $c = \text{Enc}(k, m_b)$. Now the challenger sends c to $\mathcal{A}$, and $\mathcal{A}$ sends back a bit $b' \in \{0, 1\}$ stating whether it's the first or the second experiment.

Recall, this means:

$$|\Pr[b' = 1 : \text{GAME}_{\Pi, \mathcal{A}}^{1-\text{time}}(\lambda, 0)] - \Pr[b' = 1 : \text{GAME}_{\Pi, \mathcal{A}}^{1-\text{time}}(\lambda, 1)]| \leq \text{negl}(\lambda)$$

So no adversary can distinguish the two worlds, with a probability higher than negl .

Why definition is good? Because it captures natural properties every secure SKE should have.

≡ Examples

- It should be hard to compute the secret key. *Computing the key from the ciphertext, is a way of distinguishing the two worlds, breaking the*

definition.

- It should be hard to compute the message. (*if you look at the ciphertext, it should be hard to compute the original message*)
- It should be hard to compute the first bit of the message. (*every SKE should have this property*)

On the negative side, this notion is one-time (so one key, and one message).

Let's take $c_1 = G(k) \oplus m_1$ and $c_2 = G(k) \oplus m_2$, then $c_1 \oplus c_2 = m_1 \oplus m_2$.

By knowing $c_1, c_2, m_1 \rightarrow$ I know m_2 .

Apart from this, consider again Π such that

$$\text{Enc}(k, m) = G(k) \oplus m$$

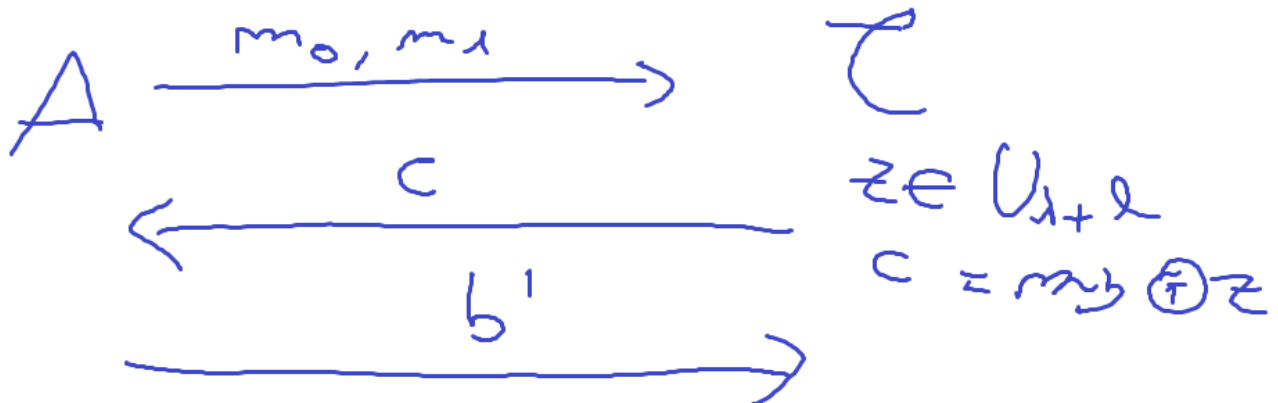
$$\text{Dec}(k, m) = G(k) \oplus c$$

✓ Theorem

If G is a PRG, the above Π is one-time computationally secure.

📖 Proof

Starting with the initial experiment $\text{GAME}(\lambda, b) \equiv \text{GAME}_{\Pi, \mathcal{A}}^{1\text{-time}}(\lambda, b)$, consider $\text{HYB}(\lambda, b)$:



I replace the output of the PRG with random. I claim that this world is indistinguishable from the real world.

Easy to see:

$$HYB(\lambda, 0) \equiv HYB(\lambda, 1)$$

Because the distribution of c is uniform and independent of b .

In the HYBRID world, no adversary can distinguish encryption of m_0 and m_1 .

On the other hand, i claim:

$$GAME(\lambda, b) \approx_c HYB(\lambda, b) \quad \forall b \in \{0, 1\}$$

By reduction: assume $\exists PPT \mathcal{A}$ such that

$$|Pr[GAME(\lambda, b) = 1] - Pr[HYB(\lambda, b) = 1]| \geq \frac{1}{poly(\lambda)}$$

for some $p(\lambda) = poly(\lambda)$.

So there exists a TM able to tell those two worlds apart, with a not negligible probability.

Then, build PPT $\mathcal{A}_{PRG}$ against G :

We have then $\mathcal{A}$, $\mathcal{A}_{PRG}$ and a challenger $\mathcal{C}_{PRG}$. $\mathcal{A}_{PRG}$ has to break G , so he receives from $\mathcal{C}_{PRG}$ a $z \in \{0, 1\}^{\lambda+l}$ with z either coming from $G(U_\lambda)$ or $U_{\lambda+l}$.

$\mathcal{A}_{PRG}$ will then tell $\mathcal{A}$ to start the algorithm. $\mathcal{A}$ gives back m_0, m_1 to $\mathcal{A}_{PRG}$, and $\mathcal{A}_{PRG}$ answers by sending $c = m_b \oplus z$ (we have that b was fixed at the beginning). $\mathcal{A}$ answers sending b' , which is passed to $\mathcal{C}_{PRG}$ by $\mathcal{A}_{PRG}$.

By inspection:

$$Pr[b' = 1 : z \in G(U_\lambda)] = Pr[b' = 1 : GAME(\lambda, b)]$$

$$Pr[b' = 1 : z \in U_{\lambda+l}] = Pr[b' = 1 : HYB(\lambda, b)]$$

$$\implies |Pr[b' = 1 : z \in G(U_\lambda)] - Pr[b' = 1 : z \in U_{\lambda+l}]| \geq \frac{1}{p(\lambda)}$$

$$\implies GAME(\lambda, 0) \approx_c HYB(\lambda, 0) \equiv HYB(\lambda, 1) \approx_c GAME(\lambda, 1)$$

$$\implies GAME(\lambda, 0) \approx_c GAME(\lambda, 1)$$

■

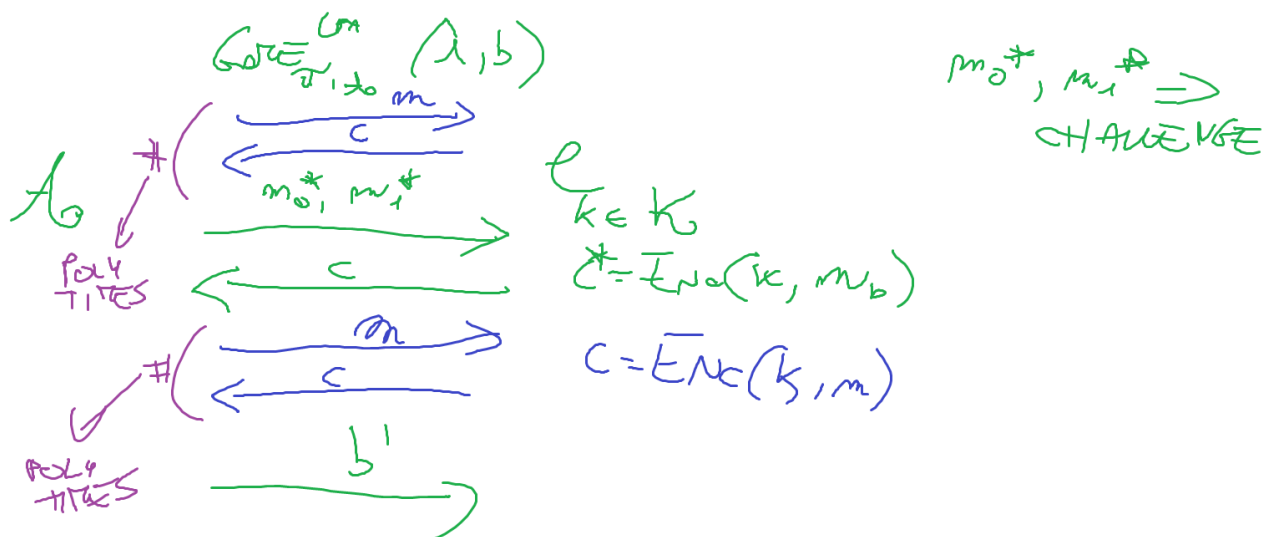
Our next goal: Chosen-plaintext attack (CPA) security.

✓ Definition

Let $\Pi = (Enc, Dec)$ be a SKE. We say Π is CPA-secure if :

$$GAME_{\Pi, \mathcal{A}}^{cpa}(\lambda, 0) \approx_c GAME_{\Pi, \mathcal{A}}^{cpa}(\lambda, 1).$$

The adversary could now do a chosen-plaintext attack, by which an adversary knows the ciphertext of a chosen message.



⚠ Observe

No deterministic SKE can achieve this.

Sanity check: Previous schema is not CPA-secure.

We need a new tool: Pseudorandom function. Let's start with the definition.

A pseudorandom function is a family of function, not like the PRG (deterministic function), which only have one input. The pseudorandom takes two inputs, one is given by the key, the other by ?.

✓ Definition

A family $F = \{F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n\}$ ($k \in \{0, 1\}^\lambda$) is a PRF family if:

We have $GAME_{F,A}^{PRF}(\lambda, 0) \approx_c GAME_{F,A}^{PRF}(\lambda, 1)$.

What is the $GAME_{F,A}^{PRF}(\lambda, b)$?

We have $\mathcal{A}$ and $\mathcal{C}$, working with $k \in \{0, 1\}^\lambda$. We have two worlds, $b = 0$ and $b = 1$. *World 0 is the actual real world.*

So $\mathcal{A}$ sends x to $\mathcal{C}$, and receives y , which could be repeated *poly* many times, and then at the end $\mathcal{A}$ has to distinguish giving b' to $\mathcal{C}$.

We have that:

$$y = \begin{cases} F_k(x) & \text{if } b = 0 \\ R(x) & \text{if } b = 1 \end{cases}$$

If $b = 1$ we use another function R , that is a random function. *Consider all possible functions that map n bits in n bits. We could think the function as a table. We consider all possible tables, and we pick one at random. R is deterministic anyways (check why).*

Let's call $\mathcal{R}$ as the set of tables that map n bits in n bits, so $R \leftarrow \mathcal{R}(\lambda, n \rightarrow n)$ (we sample R from $\mathcal{R}$).

Note: R is not efficiently computed as it takes exponential space to store it. $F(k, x)$, instead, is efficiently computed for all k, x .

Plan:

- Build a PRF.
- Use it to get CPA-secure SKE and more.

How to build a PRF?

- Practice: Many examples, like DES, AES, ... (actually, those are not just PRFs, they are so called PRPs, they are pseudorandom permutations, and they are efficiently invertible given k).
- Theory: $\text{OWF} \implies \text{PRFs} \implies \text{PRPs}$.

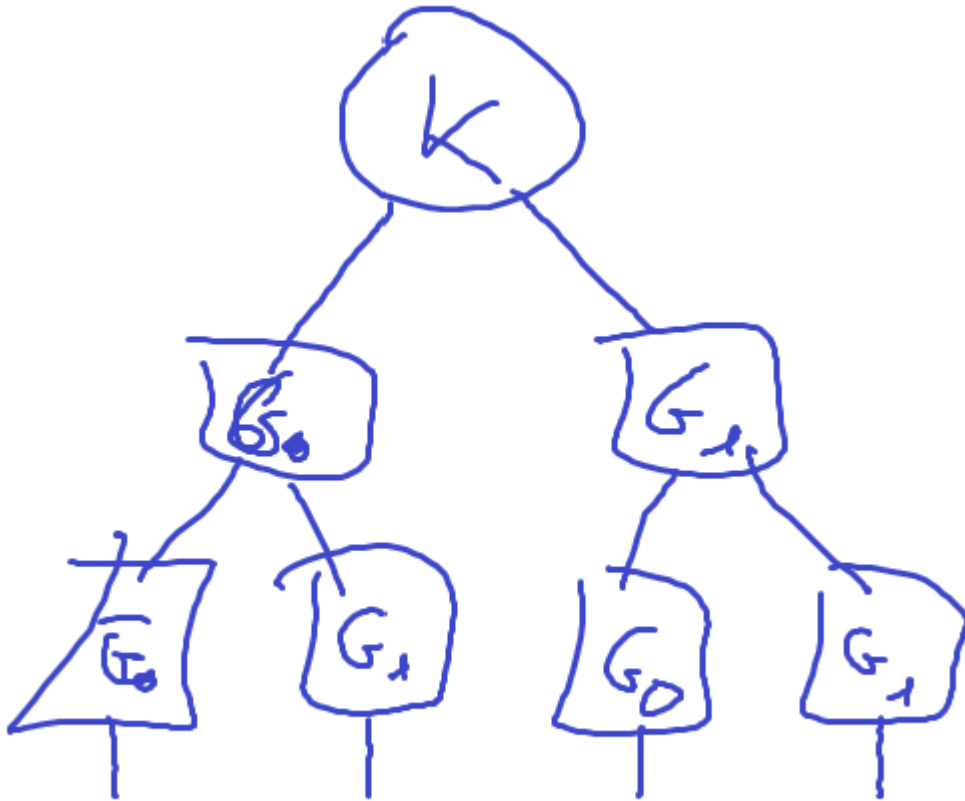
We will cover one construction of PRFs: The GGM tree. Basically, it is a proof that $\text{PRGs} \implies \text{PRFs}$. Here it is:

Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ such that $G(x) = (G_0(x), G_1(x))$. With G_0 I denote the first n bits, with G_1 the second n .

We can think of G as $F(k, x)$ for $x \in \{0, 1\}$. So x can be zero or one. y could either be $G_0(k)$ or $G_1(k)$.

We have that this configuration is $\approx_c$ to x being either 0 or 1, and y being \$, so random.

Let's say we have an input of size 2. By following the tree, I reach a final result. Let's say that we have $F(k, 01)$.



The first bit is zero, so I compute G_0 of k . The next bit is then given by 1, so I compute G_1 . The branch $k \rightarrow G_0 \rightarrow G_1$ will be $y = F_k(0, 1)$.

In general:

$$F_k(x) = G_{x_n}(G_{x_{n-1}}(\dots G_{x_1}(k)))$$

✓ Theorem

If G is a PRG, the above $F = \{F_k\}$ is a PRF.

For the proof, we will need two lemmas.

🌀 Lemma

If $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ is a PRG, then for any $t(\lambda) = \text{poly}(\lambda)$:
 $(G(k_1), \dots, G(k_t)) \approx_c (U_{2n}, \dots, U_{2n})$, with $k_1, \dots, k_t \in U_n$

Next, let $F'_k : \{0, 1\}^{n-1} \rightarrow \{0, 1\}^n$ be a PRF. Then, define $F_k(x, y) = G_x(F'_k(y))$ with $x \in \{0, 1\}, y \in \{0, 1\}^{n-1}$.

Start the PRF until $n - 1$, and do one more invocation.

🔗 Lemma

If $\{F'_k\}$ is a PRF, then $\{F_k\}$ is also a PRF.